**PROJECT REPORT**

**FOR**

**Foundations of Artificial Intelligence**

# AI BASED CAMPUS NAVIGATION SYSTEM USING A* ALGORITHM

**A PROJECT REPORT**


***Submitted by***

**2116240701492 – SHASMEEN SYED THAMEEM**

**2116240701495 – SHERIN KATHERINA D**





**October, 2026**

# BONAFIDE CERTIFICATE

Certified that this project report "**AI BASED CAMPUS NAVIGATION SYSTEM USING A* ALGORITHM**" is the bonafide work of "**SHASMEEN SYED THAMEEM AND SHERIN KATHERINA D**" who carried out the project work under my supervision.

**SIGNATURE OF THE FACULTY INCHARGE**

Submitted for the Practical Examination held on _____

**SIGNATURE OF THE INTERNAL EXAMINER**

## ABSTRACT

Finding the shortest and most convenient route between buildings is a daily challenge for students, visitors and staff in a large educational campus. This project presents an AI based Campus Navigation System that uses the A* search algorithm to compute optimal routes between campus locations. The campus is modelled as a weighted graph in which buildings and junctions are nodes and walkways are edges weighted by walking distance. A* evaluates each node using $f(n) = g(n) + h(n)$, where $g(n)$ is the cost travelled so far and $h(n)$ is the straight-line distance to the goal, which is an admissible heuristic and therefore guarantees an optimal path.

The system is implemented in Python and consists of a campus map module, an A* path-finding module and a route output module. It was tested on a 20-node, 30-edge sample campus using seven source–destination queries and compared with Dijkstra's algorithm and Breadth-First Search. A* returned the optimal route in every query while expanding 57 nodes in total against 111 for Dijkstra's algorithm, a reduction of about 49 percent. The results show that heuristic search gives accurate, efficient and explainable navigation suitable for campus kiosks and mobile applications.

# TABLE OF CONTENTS


| CHAPTER NO. | TITLE | PAGE NO. |
|---|---|---|
| | ABSTRACT | ii |
| 1 | INTRODUCTION | 1 |
| 2 | LITERATURE REVIEW | 2 |
| 3 | PROPOSED SYSTEM | 3 |
| | 3.1 ARCHITECTURE DIAGRAM | 3 |
| 4 | MODULES DESCRIPTION | 5 |
| | 4.1 CAMPUS MAP MODULE | 5 |
| | 4.2 A* PATH-FINDING MODULE | 5 |
| | 4.3 USER INTERFACE AND ROUTE OUTPUT MODULE | 6 |
| 5 | IMPLEMENTATION AND RESULTS | 7 |
| | 5.1 EXPERIMENTAL SETUP | 7 |
| | 5.2 RESULTS | 7 |
| 6 | CONCLUSION AND FUTURE WORK | 10 |
| 7 | REFERENCES | 11 |

# CHAPTER 1
# INTRODUCTION

Large educational campuses contain many academic blocks, laboratories, hostels, libraries, canteens and sports facilities spread over a wide area. New students, visiting parents, guests and examiners often find it difficult to locate a destination quickly, and printed maps or signboards do not suggest the best route from the place where a person currently stands. A digital navigation aid that can answer the question “how do I get from here to there?” is therefore a practical and useful application of Artificial Intelligence.

Route finding is a classical problem in AI that can be solved by searching a state space. In the campus context, every building or junction is a state, every walkway is an action, and the walking distance is the cost of that action. Uninformed algorithms such as Breadth-First Search explore the map blindly, while informed algorithms use domain knowledge to focus the search towards the goal. The A* algorithm combines the actual cost already travelled with an estimate of the remaining cost and is both complete and optimal when the estimate never over-states the true cost.

**Problem Statement.** To design and implement an AI based system that accepts a source and a destination inside a campus and returns the shortest walking route using the A* algorithm, along with the total distance and estimated walking time.

**Objectives.** The objectives of the project are:

- to model the campus as a weighted graph of locations and walkways;
- to implement A* search with an admissible Euclidean-distance heuristic;

- to compare A* with Dijkstra's algorithm and Breadth-First Search in terms of route cost and search effort; and
- to present the route in a simple step-by-step and map-based form that any user can follow.

**Scope.** The system works on a static map of one campus and considers walking distance as the only cost. Live crowd data, indoor floor-level navigation and accessibility constraints are outside the present scope and are listed as future work.

# CHAPTER 2
# LITERATURE REVIEW

Shortest-path search has a long history in computer science. Dijkstra (1959) proposed a method that finds minimum-cost paths in a weighted graph by always expanding the unvisited node with the smallest accumulated cost. The method is optimal but uninformed, because it expands nodes in every direction around the source, even those leading away from the destination.

Hart, Nilsson and Raphael (1968) introduced the A* algorithm, which adds a heuristic estimate h(n) of the remaining cost to the accumulated cost g(n). They proved that if h(n) never over-estimates the true remaining cost, the first path found to the goal is optimal, and that no other algorithm using the same heuristic information expands fewer nodes. Pearl (1984) and Russell and Norvig (2021) discuss admissibility and consistency of heuristics in detail and show how a good heuristic reduces the effective branching factor of the search.

Goldberg and Harrelson (2005) combined A* search with landmark-based lower bounds to speed up queries on very large road networks, and Delling et al. (2009) and Bast et al. (2016) surveyed route-planning techniques used in transportation networks, from goal-directed search to hierarchical and pre-processing methods. These works confirm that A* is the foundation of most practical route planners. Cormen et al. (2022) present the underlying graph representations and priority-queue data structures used to implement such algorithms efficiently.

Most of the above work addresses city-scale or national road networks. Campus navigation is a smaller problem, but it has its own needs:

pedestrian paths rather than roads, a limited number of well-known landmarks, and users who need clear instructions on a phone or kiosk. This project applies the established A* technique to this setting and measures its benefit over uninformed search on a campus-sized graph.

# CHAPTER 3
# PROPOSED SYSTEM

The proposed system represents the campus as an undirected weighted graph G = (V, E). Each vertex in V is a campus location with planar coordinates (x, y) in metres, and each edge in E is a walkway whose weight is its walking length. For a requested source s and destination t, the A* engine maintains an open list ordered by the evaluation function

$$f(n) = g(n) + h(n)$$

where g(n) is the cost of the best known path from s to n and h(n) is the straight-line (Euclidean) distance from n to t. Because a walkway can never be shorter than the straight line between its end points, h(n) never over-estimates the real remaining cost; it is therefore admissible and also consistent, and A* returns a shortest path. The algorithm proceeds as follows:

1. Place the source in the open list with g = 0 and f = h(source).
2. Remove the node with the lowest f value from the open list and mark it closed.
3. If the node is the destination, reconstruct the path using parent links and stop.
4. For each neighbour, compute the new cost g' = g(current) + edge length; if g' is lower than any earlier cost for that neighbour, update its parent and insert it in the open list with f = g' + h.
5. Repeat from step 2; if the open list becomes empty, report that no route exists.

## 3.1 ARCHITECTURE DIAGRAM

Figure 3.1 shows the architecture of the system. The user enters a source and a destination, which are validated and mapped to graph nodes. The A* search engine takes the graph from the campus map module and the estimates from the heuristic module, and returns the best node sequence. The path reconstruction block converts this sequence into total distance, estimated walking time and readable directions that are shown to the user.

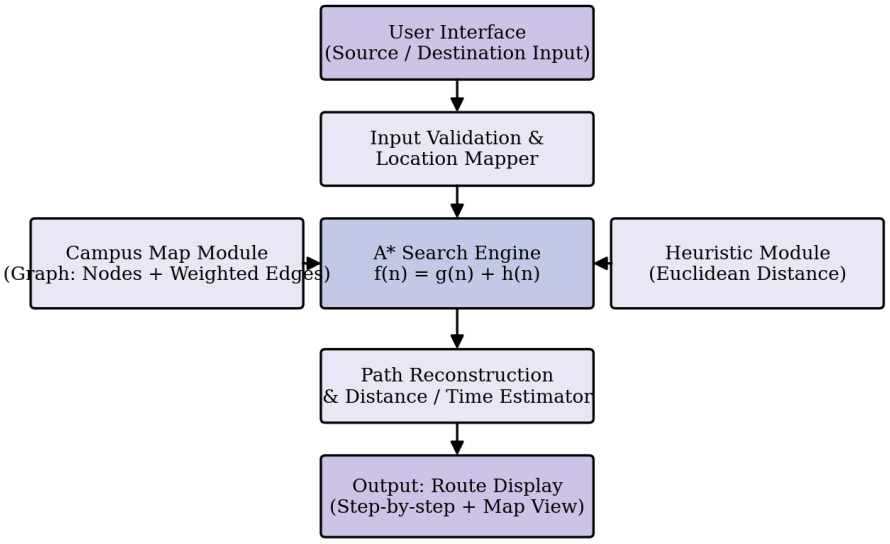



**Figure 3.1: Architecture of the AI based Campus Navigation System**

# CHAPTER 4
# MODULES DESCRIPTION

The system is divided into three modules, each with a single responsibility, so that the campus data, the search logic and the user presentation can be changed independently.

## 4.1 CAMPUS MAP MODULE

This module stores the campus as an adjacency list. Every node holds a location name and its (x, y) coordinates; every edge holds the walking distance between two adjacent locations. The sample map used in this project has 20 nodes, including the Main Gate, Administrative Block, Library, academic blocks, Seminar Hall, Canteen, Medical Centre, hostels, Placement Cell, Research Lab and Sports Ground, joined by 30 walkways. Edge weights are set equal to or greater than the straight-line distance between the end points to represent bends in the paths. To adapt the system to another campus, only this data needs to be replaced. Figure 4.1 shows the graph model.

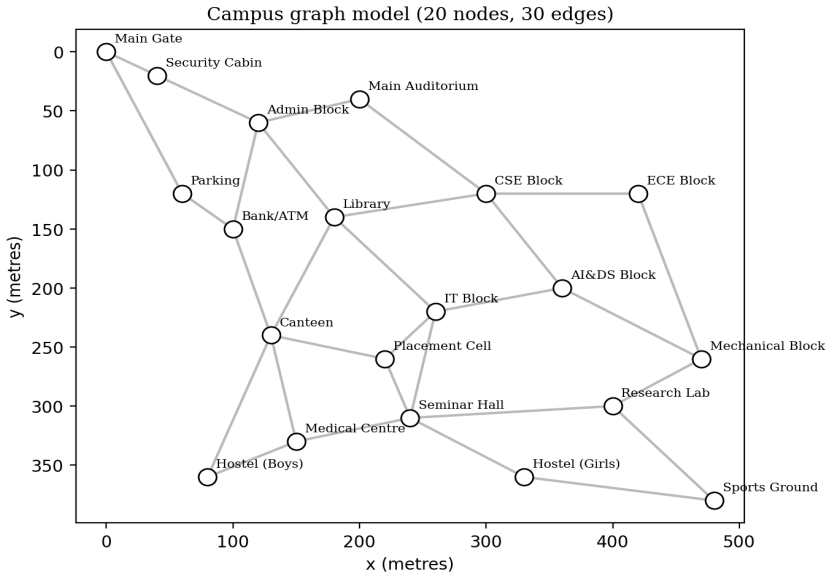



**Figure 4.1: Campus graph model**

### 4.2 A* PATH-FINDING MODULE

This module is the core of the system. It uses a priority queue (binary heap) as the open list, a dictionary of best-known costs g(n), a dictionary of parent links and a closed set. The heuristic function returns the Euclidean distance from a node to the goal. With the heuristic switched off (h = 0) the same code behaves as Dijkstra's algorithm, which makes the comparison in Chapter 5 fair. Breadth-First Search is implemented separately as a baseline that ignores edge weights. The module returns the path, its cost and the number of nodes expanded.

### 4.3 USER INTERFACE AND ROUTE OUTPUT MODULE

This module accepts the source and destination names, checks that both exist on the map and passes them to the search module. It then converts the returned node list into an ordered set of directions such as "Walk from Main Gate to Security Cabin", adds the total distance and estimates the walking time at an average pace of about 1.3 m/s, and highlights the route on the campus map as shown in Chapter 5. The same output can be used by a command-line tool, a web page or a mobile application.

# CHAPTER 5
# IMPLEMENTATION AND RESULTS

## 5.1 EXPERIMENTAL SETUP

The system was implemented in Python 3 using the standard library modules heapq and math, and Matplotlib for drawing the map. The sample campus has 20 locations and 30 walkways. Seven source–destination queries (Q1 to Q7) covering short, medium and long trips were selected. For each query, A*, Dijkstra's algorithm and Breadth-First Search were run, and the route cost (in metres) and the number of nodes expanded were recorded. The core of the A* implementation is given below.

```
def a_star(graph, coords, start, goal):
    h = lambda n: euclid(coords[n], coords[goal])   # admissible
heuristic
    open_list = [(h(start), 0, start)]               # (f, g, node)
    best_g, parent, closed = {start: 0}, {start: None}, set()
    expanded = 0
    while open_list:
        f, g, u = heapq.heappop(open_list)
        if u in closed:
            continue
        closed.add(u); expanded += 1
        if u == goal:
            return reconstruct(parent, u), g, expanded
        for v, w in graph[u].items():
            ng = g + w
            if ng < best_g.get(v, float('inf')):
                best_g[v], parent[v] = ng, u
                heapq.heappush(open_list, (ng + h(v), ng, v))
    return None, float('inf'), expanded
```

**Listing 5.1: A* search implementation**

## 5.2 RESULTS

Table 5.1 lists the route cost and search effort for each query. In all seven queries A* and Dijkstra's algorithm returned routes of identical, optimal cost, confirming that the Euclidean heuristic does not compromise optimality. Breadth-First Search minimises the number of hops rather than distance, and for Q3 it returned a route of 754.2 m against the optimal 745.2 m.

| No. | Source → Destination | A* cost (m) | A* exp. | Dijkstra exp. | BFS exp. | BFS cost (m) |
|---|---|---|---|---|---|---|
| Q1 | Main Gate → Library | 253.1 | 4 | 7 | 7 | 253.1 |
| Q2 | Main Gate → AI&DS Block | 457.8 | 8 | 14 | 14 | 457.8 |
| Q3 | Main Gate → Sports Ground | 745.2 | 16 | 20 | 20 | 754.2 |
| Q4 | Parking → Hostel (Girls) | 437.8 | 6 | 15 | 17 | 437.8 |
| Q5 | Admin Block → Research Lab | 515.7 | 11 | 19 | 19 | 515.7 |
| Q6 | Canteen → ECE Block | 382.3 | 7 | 18 | 16 | 382.3 |
| Q7 | Hostel (Boys) → Mechanical Block | 447.2 | 5 | 18 | 15 | 447.2 |

**Table 5.1: Comparison of A*, Dijkstra and BFS on seven queries (exp. = nodes expanded)**

Across the seven queries A* expanded 57 nodes in total, compared with 111 for Dijkstra's algorithm and 108 for Breadth-First Search. This is a reduction of about 49 percent against Dijkstra, because A* directs the search towards the destination instead of spreading evenly around the source. The saving grows for destinations that lie far from the source and for larger maps. The run time of every query was well below one millisecond for all three methods on this small graph, so execution time is not a deciding factor at this scale; the benefit of A* will become significant on larger campuses.

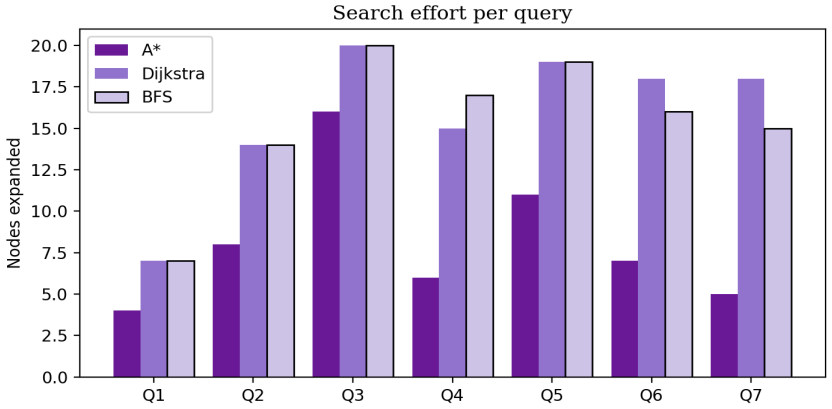



**Figure 5.1: Nodes expanded by each algorithm for queries Q1 to Q7**

As an example, the A* route for Q3 from the Main Gate to the Sports Ground is: Main Gate → Parking → Bank/ATM → Canteen →

Placement Cell → Seminar Hall → Hostel (Girls) → Sports Ground, with a total length of 745.2 m, which corresponds to roughly 9.5 minutes of walking at 1.3 m/s. Figure 5.2 shows this route highlighted on the campus map.

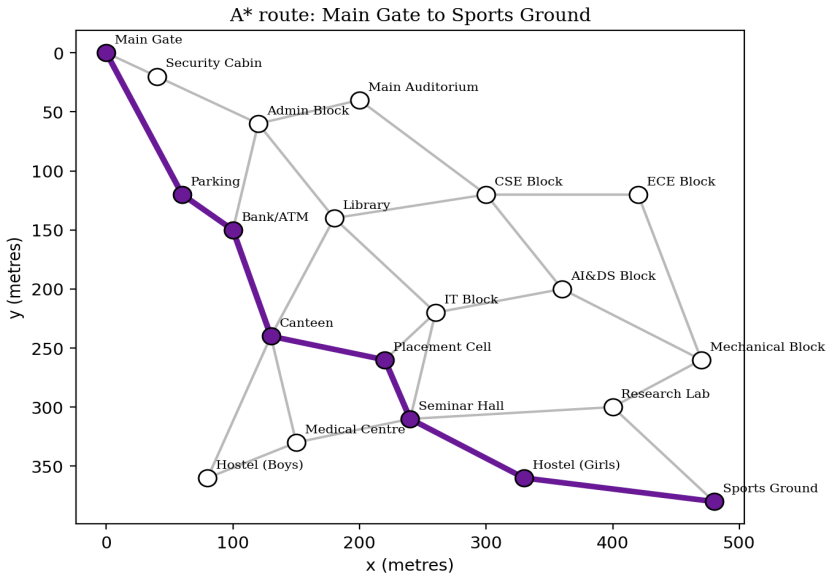



**Figure 5.2: Shortest route found by A* from Main Gate to Sports Ground**

# CHAPTER 6
# CONCLUSION AND FUTURE WORK

This project designed and implemented an AI based Campus Navigation System that models a campus as a weighted graph and finds routes with the A* algorithm. Experiments on a 20-node sample campus showed that A* always found the optimal route and needed about 49 percent fewer node expansions than Dijkstra's algorithm, while Breadth-First Search could not guarantee the shortest distance. The modular design allows the map, the search logic and the user interface to be modified independently, and the output is easy for a user to understand and follow.

The system can be extended in the following ways:

- replacing the sample map with the surveyed map of the actual campus, using GPS or OpenStreetMap data;
- adding a web or mobile interface with live location tracking and voice guidance;
- including accessibility options such as ramp-only routes and avoidance of stairs;
- adding dynamic costs for crowding, construction work, weather or class timings; and
- supporting indoor navigation between floors and rooms, and multi-stop route planning.

# CHAPTER 7

# REFERENCES


Bast, H., Delling, D., Goldberg, A., Müller-Hannemann, M., Pajor, T., Sanders, P., Wagner, D. and Werneck, R.F. (2016) 'Route planning in transportation networks', in Algorithm Engineering, Lecture Notes in Computer Science, Vol.9220, Springer, pp.19-80.

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) Introduction to Algorithms, 4th edn, MIT Press, Cambridge, MA.

Delling, D., Sanders, P., Schultes, D. and Wagner, D. (2009) 'Engineering route planning algorithms', in Algorithmics of Large and Complex Networks, Lecture Notes in Computer Science, Vol.5515, Springer, pp.117-139.

Dijkstra, E.W. (1959) 'A note on two problems in connexion with graphs', Numerische Mathematik, Vol.1, No.1, pp.269-271.

Goldberg, A.V. and Harrelson, C. (2005) 'Computing the shortest path: A* search meets graph theory', Proceedings of the 16th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pp.156-165.

Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A formal basis for the heuristic determination of minimum cost paths', IEEE Transactions on Systems Science and Cybernetics, Vol.4, No.2, pp.100-107.

Pearl, J. (1984) Heuristics: Intelligent Search Strategies for Computer Problem Solving, Addison-Wesley, Reading, MA.

Russell, S. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach, 4th edn, Pearson, Hoboken, NJ.